

Introduction to Docker

Using Docker Containers:

What is a Container (in terms of running code like Python)?

A **container** is a **running instance** of a Docker image.

If an image is like a Python script with all its dependencies packaged in a zip file, then a container is like **unpacking that zip file and executing the script in its own isolated environment**.

You can think of it as:

- A **temporary lightweight virtual environment**.
- It **starts** when you run the image.
- It **stops** when the main process (like python script.py) finishes.

The container includes:

- The base OS (e.g. Ubuntu)
- Python installed
- Your .py file
- Any libraries (e.g. NumPy, pandas)

Once the script finishes running, the container **stops**, but still exists on your system in a non-running state until you manually remove it.

1. What is Docker

Docker is a platform used for containerization of applications. It allows the applications to be run on any machine as separate parts, without any complications with the dependencies, as it uses the main host OS kernel.

2. What is a Docker Daemon

The *Docker Daemon (dockerd)* is a *background service* that:

- **Manages containers** (builds, runs, stops, removes).
- **Handles Docker objects** like images, networks, and volumes.
- **Listens for Docker API requests** (usually sent via the Docker CLI or Docker Compose).

3. What is a Docker Image

A *Docker image* is a *template* used to create Docker containers.

It's a **blueprint** that contains:

- The **application code**.
- All **dependencies** (libraries, binaries).

- A **runtime environment** (e.g., Ubuntu, Python).
- Configuration files and default commands.

It is read-only, so when starting a container with **docker run <image-name>**, the docker creates a container from that template.

4. Interactive and Detached Containers

When you run a container with **-it**, you're telling Docker to:

- **-i** → Keep STDIN open (interactive).
- **-t** → Allocate a TTY (a terminal session).

Purpose: Allows you to interact with the container via a shell (e.g., bash).

Use case: Running a container where you want to type commands directly

When you run a container with **-d**, you're telling Docker to:

- Run the container in the **background** (detached mode).
- It won't attach to your terminal.

Use case: Running a container that should keep running independently, like a web server

Usage	Command
Start a container	<code>docker run <image-name></code>
Start an interactive container	<code>docker run -it <image-name></code>
Start a detached container	<code>docker run -d <image-name></code>
List running containers	<code>docker ps</code>
Stop a container	<code>docker stop <container-id></code>

5. Named Containers and Logs

docker run --name <container-name> <image-name>

Starts a container and gives it a custom name.

Why? Makes it easier to manage or refer to specific containers, especially if you're running multiple ones.

docker ps -f "name=<container-name>"

Filters the list of running containers based on the name.

Why? Useful for finding a specific container among many.

6. Viewing Logs

docker logs <container-id>

Shows the output (logs) that the container has produced.

Why? Helps you debug or monitor what the container is doing or whether it ran correctly.

docker logs -f <container-id>

Follows the logs in real time (like tail -f).

Why? Useful for monitoring a running service live, e.g., seeing web server requests as they come in.

CTRL+C (In PowerShell)

Stops the real-time log viewing (but doesn't stop the container itself).

Why? Just a key shortcut to exit the live view when you're done watching logs.

7. Cleaning Up

docker container rm <container-id>

Removes a **stopped** container permanently.

Why? Keeps your system clean — containers take up space, and you don't want to leave unused ones piling up.

Usage	Command
Start container with a name	<code>docker run --name <container-name> <image-name></code>
Filter running container on name	<code>docker ps -f "name=<container-name>"</code>
See existing logs for container	<code>docker logs <container-id></code>
See live logs for container	<code>docker logs -f <container-id></code>
Exit live log view of container	CTRL+C
Remove stopped container	<code>docker container rm <container-id></code>

8. Managing Images and Containers

docker pull <image-name>

Downloads an image from Docker Hub (or another registry) to your machine.

Why? You need the image locally before you can run a container from it.

docker pull <image-name>:<image-version>

Downloads a **specific version (tag)** of an image.

Why? Different versions may have:

- Different library versions.
- Different Python versions.
- Security patches or behavior changes.

Use this if your code requires a specific environment (e.g., python:3.9 vs python:3.11).

docker images

Lists all images currently available (downloaded) on your machine.

Why? Helps you see what you already have so you don't pull duplicates or forget what's installed.

docker image rm <image-name>

Deletes an image.

Why? To free up space or clean up unused images.

If a container is using that image, Docker won't let you delete it unless the container is removed first.

9. Cleaning Up

docker container prune

Removes all **stopped containers** (containers that have finished running).

Why? These are no longer doing anything and just take up disk space.

A "stopped container" is like a paused or terminated Python process — it ran, then stopped, but its resources (logs, settings, etc.) still exist on disk.

docker image prune -a

Removes **all unused images** (not tied to running containers).

Why? Frees up space. The -a flag means **remove all**, not just dangling (intermediate) ones.

Usage	Command
Pull an image	docker pull <image-name>
Pull a specific version of an image	docker pull <image-name>:<image-version>
List all local images	docker images
Remove an image	docker image rm <image-name>
Remove all stopped containers	docker container prune
Remove all images	docker image prune -a

Writing Your Own Docker Images

10. Managing and Sharing Docker Images

Once you build your own image, you can store, share, or move it using registries or local files.

docker pull <private-registry-url>/<image-name>

Downloads an image from a **private registry** instead of Docker Hub.

Why? Many companies host internal images privately (e.g., for security or proprietary apps)

docker tag <old-name> <new-name>

Gives an image a new name or tag.

Example: `docker tag myapp myregistry.com/myapp:v1`

Why? Required before pushing to a different registry or naming versions explicitly.

docker image push <image-name>

Uploads your image to a registry (like Docker Hub or a private one).

Why? So others can pull and run your image on their systems. Useful for sharing or deployment.

docker login <private-registry-url>

Authenticates you to a private registry using your credentials.

Why? Required before pushing or pulling from a private registry.

docker save -o <file-name> <image-name>

Saves an image to a .tar file on disk.

Why? Useful for backing up or transferring images offline (e.g., between air-gapped systems).

docker load -i <file-name>

Loads an image from a .tar file created with docker save.

Why? Restores a saved image into your local Docker system.

Usage	Command
Pull image from private registry	<code>docker pull <private-registry-url>/<image-name></code>
Name an image	<code>docker tag <old-name> <new-name></code>
Push an image	<code>docker image push <image-name></code>
Login to private registry	<code>docker login <private-registry-url></code>
Save image to file	<code>docker save -o <file-name> <image-name></code>
Load image from file	<code>docker load -i <file-name></code>

11. Writing and Building a Dockerfile

A **Dockerfile** is a plain text file with instructions for building a Docker image.

Each instruction adds a layer to the image. It defines what should be **installed**, **copied**, and **run** inside the container.

FROM <image-name>

Sets the **base image** to build on (e.g., ubuntu, python:3.11).

Why? You always start with an existing image as your foundation.

RUN <shell-command>

Executes a command during the build process.

Example:

RUN apt-get update && apt-get install -y python3

Why? Used to install packages, set permissions, or configure the environment.

Note: Use flags like -y to avoid prompts that would pause the build (non-interactive build required).

Building Docker Images from a Dockerfile

Once your Dockerfile is ready, you build it using the docker build command.

docker build /location/to/Dockerfile

Builds an image from a Dockerfile in a specific path.

Why? You might keep your Dockerfile in a separate folder.

docker build .

Builds an image using the Dockerfile in the **current directory**.

Why? Common and convenient when you're working in a local project folder.

docker build -t <image-name> .

Adds a tag (-t) to name your image during the build.

Example:

docker build -t myapp .

Why? Naming the image makes it easier to run, push, or manage later.

Usage	Dockerfile Instruction
Start a Dockerfile from an image	FROM <image-name>
Add a shell command to image	RUN <valid-shell-command>
Make sure no user input is needed for the shell-command.	RUN apt-get install -y python3

Usage	Shell Command
Build image from Dockerfile	docker build /location/to/Dockerfile
Build image in current working directory	docker build .
Choose a name when building an image	docker build -t first_image .

12. Copying Files into Docker Images

You often need to copy your application code or other files into the container during the image build. Docker uses the COPY instruction inside the Dockerfile to do this.

COPY <src-path-on-host> <dest-path-on-image>

Copies a single file from your local system into the image.

Example:

COPY app.py /app/app.py

Why? The container needs access to your code to execute it.

COPY <src-folder> <dest-folder>

Copies an entire folder and its contents.

Example:

COPY ./mycode /app/mycode

Why? Useful when your app has multiple files or a project structure.

Note:

You **cannot copy files from outside the build context**, like:

COPY ../somefile /app/

Why? For security and consistency, Docker only allows access to files within the build context (i.e., the folder where docker build is run).

Keeping Images Small and Clean

You can chain multiple commands in a single RUN instruction to avoid leaving behind unnecessary files (like .zip files after extracting).

Example:

```
RUN curl <url> -o /tmp/file.zip \  
  && unzip /tmp/file.zip -d /app \  
  && rm /tmp/file.zip
```

Why?

- Fewer layers in the image.
- Smaller final image size.
- Better performance and portability.

Usage	Dockerfile Instruction
Copy files from host to the image	COPY <src-path-on-host> <dest-path-on-image>
Copy a folder from host to the image	COPY <src-folder> <dest-folder>
We can't copy from a parent directory where we build a Dockerfile	COPY ../<file-in-parent-directory> /

Keep images small by downloading, unzipping, and cleaning up in a single RUN instruction:

```
RUN curl <file_download_url> -o <destination_directory> \  
  && unzip <destination_directory>/<filename>.zip -d <unzipped-directory> \  
  && rm <destination_directory>/<filename>.zip
```

RUN curl

https://assets.datacamp.com/production/repositories/6082/datasets/31a5052c6a5424cbb8d939a7a6eff9311957e7d0/pipeline_final.zip -o /pipeline_final.zip

RUN unzip /pipeline_final.zip

RUN rm /pipeline_final.zip